

Analyse du besoin – Projet NSY103 « Mon D.O.S. »

CNAM Toulouse (IPST) – Module NSY103

Auteurs : Enzo FOURNET & Soltan HAMADOUCHE

1. Objectif et besoin utilisateur

Le projet consiste à réaliser en langage C un mini système d'exploitation centré sur un émulateur de système de gestion de fichiers (SGF) et un mini-interpréteur de commandes de type Bash. L'utilisateur doit pouvoir saisir des commandes dans un terminal, manipuler une arborescence de fichiers et de répertoires, puis consulter le résultat des opérations dans un programme d'affichage distinct.

Notre objectif principal est de reproduire, de manière simplifiée mais cohérente, la chaîne de traitement d'un système Unix : le Shell reçoit la commande et l'envoie au Noyau ; le Noyau l'exécute à l'aide des primitives du SGF ; le SGF lit ou modifie l'espace de stockage simulé ; le résultat revient ensuite au Noyau puis au Shell, qui le transmet à l'Écran. Nous souhaitons conserver une architecture modulaire afin de pouvoir développer, tester et maintenir chaque partie indépendamment.

Pour respecter le périmètre minimal demandé, nous prendrons en charge la navigation dans l'arborescence, la création et la suppression de fichiers et répertoires, la lecture et l'écriture de données, la copie et le déplacement de fichiers, l'affichage de l'occupation du SGF, les arguments simples comme « ls -l » et une redirection telle que « echo "texte" > fichier ». Nous pourrions ajouter les droits dans un second temps en distinguant uniquement « utilisateur » et « autres ».

2. Architecture générale

Nous avons retenu une architecture qui sépare quatre responsabilités :

- **Shell** : processus frontal en attente des commandes. Il affiche le prompt, lit la ligne saisie, effectue l'analyse syntaxique, construit une requête structurée et l'envoie au Noyau. Il reçoit ensuite la réponse du Noyau et la transmet à l'Écran. Le Shell ne doit pas accéder directement aux inodes, aux bitmaps ou au fichier disque.
- **Noyau** : processus fils créé par le Shell. Il reçoit les requêtes, sélectionne l'opération à exécuter, appelle les fonctions du SGF, transforme les codes de retour en résultat exploitable et renvoie ce résultat au Shell. Il conserve également l'inode

du répertoire courant utilisé pour résoudre les chemins relatifs ; la commande `cd` modifie cet état virtuel sans modifier le répertoire courant du système hôte.

- **SGF** : bibliothèque de fonctions appelée par le Noyau et responsable de la structure logique du système de fichiers : superbloc, inodes, blocs de données, répertoires, résolution des chemins, allocation et libération des ressources. Elle lit et écrit les données dans le disque virtuel.
- **Écran** : programme C indépendant, distinct du Shell et non fils de celui-ci. Il reçoit uniquement les messages envoyés par le Shell et affiche les résultats des commandes ainsi que les erreurs. En mode natif, il peut être lancé dans un terminal dédié ; en mode conteneurisé, son démarrage sera automatisé.

Nous retenons donc le flux logique suivant : Écran $\leftarrow$ Shell $\leftrightarrow$ Noyau $\leftrightarrow$ SGF $\leftrightarrow$ disque virtuel (Data). L'utilisateur saisit ses commandes dans le Shell ; les résultats reviennent jusqu'au Shell avant d'être transmis à l'Écran.

Nous prévoyons deux modes d'exécution : directement sur la machine hôte ou à l'intérieur d'un conteneur. La conteneurisation constituera uniquement un environnement reproductible et ne modifiera pas l'architecture interne : le Shell créera toujours le processus Noyau avec `fork()`, puis communiquera avec lui par pipes et avec l'Écran par FIFO. Nous fournirons un point d'entrée unique qui démarrera automatiquement les programmes nécessaires afin que l'utilisateur n'ait qu'une seule commande Docker à exécuter, sans ouvrir plusieurs shells dans le conteneur.

3. Communications interprocessus

Pour répondre à la demande du sujet concernant les communications entre les processus, nous avons retenu les moyens suivants :

- **Shell $\leftrightarrow$ Noyau** : deux tubes anonymes (pipes), adaptés à la relation père/fils créée par `fork()`. Un pipe transporte les commandes du Shell vers le Noyau et un second transporte les réponses dans le sens inverse. Une structure commune décrivant une commande contiendra notamment un type de commande, le nombre d'arguments et les arguments sérialisés dans des buffers de taille définie. Aucun pointeur mémoire ne sera transmis directement entre les processus.
- **Shell $\rightarrow$ Écran** : un tube nommé FIFO. L'Écran étant un programme autonome et non un fils du Shell, le FIFO permet une communication simple entre processus indépendants. Après l'exécution d'une commande, le Noyau renvoie le résultat au Shell par le pipe de retour ; le Shell transmet ensuite ce résultat à l'Écran. Nous pourrions étudier une socket Unix comme alternative, mais elle n'est pas nécessaire pour satisfaire le besoin minimal.

Nous définirons le protocole IPC dans un fichier commun afin d'éviter que chaque module interprète différemment les messages. Les échanges Shell $\leftrightarrow$ Noyau reposeront sur des

structures connues des deux processus, tandis que le flux Shell → Écran transportera les résultats à afficher. Nous réaliserons les échanges Noyau ↔ SGF par des appels de fonctions et des valeurs de retour, et les échanges SGF ↔ Data par des lectures et écritures dans disque.img.

4. Conception du système de fichiers

Nous concevrons le SGF en nous inspirant du modèle Unix/Linux et en utilisant un seul niveau d'indirection, conformément au sujet. Nous représenterons le stockage persistant par un fichier binaire, par exemple « disque.img », qui simulera un disque dur.

Nous découperons le disque virtuel en blocs de taille fixe. Nous avons retenu une taille de 4096 octets par bloc, car elle simplifie le calcul des positions, l'allocation et la réutilisation de l'espace. Nous organiserons le fichier en zones successives :

- Superbloc ;
- Bitmap des inodes ;
- Bitmap des blocs de données ;
- Table des inodes ;
- Blocs de données.

Le superbloc contiendra les informations globales : taille d'un bloc, nombre total et libre de blocs, nombre total et libre d'inodes et position des différentes zones.

Chaque inode décrira un fichier ou un répertoire. Il contiendra au minimum : un identifiant, un type, la taille, les droits, plusieurs pointeurs vers des blocs directs et un pointeur vers un bloc d'indirection simple. Ce bloc indirect contiendra lui-même une liste de numéros de blocs de données. Les structures persistantes utiliseront autant que possible des types de taille fixe, comme `uint32_t` ou `uint16_t`, afin de maîtriser leur représentation dans disque.img.

Deux bitmaps assureront l'allocation : 0 signifie « libre » et 1 « utilisé ». La création d'un fichier réserve un inode et les blocs nécessaires ; la suppression libère ses blocs et son inode, permettant de réutiliser immédiatement l'espace.

Nous représenterons un répertoire comme un fichier particulier contenant des couples « nom → numéro d'inode ». Nous créerons la racine « / » lors du formatage du disque et utiliserons les entrées « . » et « .. » pour la navigation. Nous centraliserons la résolution des chemins dans une fonction `resolve_path()`, chargée de convertir un chemin absolu ou relatif en inode et utilisée par la majorité des primitives et des commandes.

Au premier lancement, nous utiliserons une fonction `fs_format()` pour créer et initialiser le disque virtuel. Aux lancements suivants, `fs_mount()` chargera le système existant. Nous considérons la persistance entre deux exécutions comme un critère fonctionnel important.

5. API du SGF et commandes du Shell

Nous conserverons les primitives demandées par le sujet :

- `_mycreat(nom, mode)` : crée un fichier et retourne son inode ;
- `_myopen(nom, mode)` : ouvre ou crée un fichier ;
- `_myclose(inode)` : ferme un fichier ;
- `_myread(inode, buffer, nombre)` : lit des octets ;
- `_mywrite(inode, buffer, nombre)` : écrit des octets ;
- `_mkdir(nom)` : crée un répertoire ;
- `_rmdir(nom)` : supprime un répertoire vide ;
- `_unlink(nom)` : supprime une entrée de répertoire.

Nous ajouterons également des fonctions internes pour structurer l'implémentation : `allocate_inode()`, `free_inode()`, `allocate_block()`, `free_block()`, `read_inode()`, `write_inode()`, `resolve_path()`, `directory_find()`, `directory_add()` et `directory_remove()`.

Nous implémenterons au minimum dans le mini-Shell les commandes `ls`, `ls -l`, `mkdir`, `rmdir`, `cd`, `cp`, `rm`, `mv`, `cat`, `echo "texte" > fichier` et `df`. Nous ajouterons également `pwd`, `help` et `exit` afin de faciliter l'utilisation et les tests. Notre parser distinguera le nom de la commande, ses arguments, les chaînes entre guillemets et les opérateurs simples de redirection.

La commande saisie ne sera pas interprétée directement par le SGF. Le Shell transformera la ligne en une structure de requête contenant un type de commande et ses arguments. Le nom textuel de la commande sera ainsi interprété une seule fois par le Shell avant l'envoi au Noyau.

Nous utiliserons dans le Noyau une table de dispatch associant chaque type de commande à une fonction C via un pointeur de fonction. Cette organisation évitera une longue succession de `if/else` ou de `switch` et permettra d'ajouter une commande sans modifier la logique générale du Noyau. La table pourra également contenir des informations telles que le nom de la commande, le nombre d'arguments attendu ou son texte d'aide.

Le traitement que nous mettrons en place suivra donc le principe suivant :

Commande texte → parsing du Shell → requête structurée → table de dispatch du Noyau → fonction de commande → primitives du SGF → réponse au Noyau → retour au Shell → affichage par l'Écran.

Cette séparation permet de conserver indépendants le parsing, la logique des commandes et l'accès au système de fichiers.

6. Gestion des erreurs, organisation du code et validation

Nous définirons des codes d'erreur communs retournés par les fonctions du SGF : succès, fichier introuvable, élément déjà existant, espace insuffisant, mauvais type d'objet, répertoire non vide, chemin invalide, erreur d'entrée/sortie et permission refusée. Nous construirons les messages destinés à l'utilisateur au niveau du Shell et de l'Écran.

Nous proposerons également une exécution dans un conteneur afin de fournir un environnement de compilation et d'exécution reproductible. Nous placerons le fichier disque.img dans un volume persistant, ce qui permettra de recréer le conteneur sans perdre le contenu du SGF. Nous fournirons un script de démarrage unique qui lancera automatiquement le programme Écran puis le Shell, lequel créera le processus Noyau. L'utilisateur n'aura ainsi qu'une seule commande Docker à exécuter et n'aura pas à ouvrir plusieurs shells dans le conteneur. Nous conserverons toutefois une exécution native afin que le projet puisse également être compilé et lancé sans conteneur.

Nous organiserons le code par responsabilités, par exemple :

- **shell/** : boucle principale et parser ;
- **kernel/** : création du processus, table de dispatch et implémentation des commandes ;
- **fs/** : disque, superbloc, inodes, blocs, répertoires, chemins ;
- **ipc/** : pipes et FIFO ;
- **screen/** : programme d'affichage ;
- **common/** : structures de messages, types et codes d'erreur.

Nous considérerons le système comme fonctionnel lorsqu'un scénario complet pourra être exécuté : créer un répertoire, s'y déplacer, créer et écrire un fichier, le lire, le copier, le déplacer puis le supprimer ; afficher l'espace utilisé avec df ; fermer et relancer l'application en conservant les données ; enfin vérifier que l'espace libéré après suppression peut être réutilisé. Nous vérifierons également que le mode conteneurisé démarre automatiquement l'Écran et le Shell à partir d'une seule commande, que le Shell crée ensuite le Noyau avec fork() et que disque.img reste conservé dans le volume persistant.

7. Répartition proposée des tâches

Nous proposons la répartition suivante comme base de travail ; elle pourra évoluer au cours du développement en fonction de notre avancement.

- Enzo FOURNET – SGF bas niveau et environnement :
conception de disque.img, superbloc, bitmaps, blocs, inodes, formatage, compilation, Makefile, conteneurisation et script de démarrage unique.
- Soltan HAMADOUCHE – SGF haut niveau et Shell :
répertoires, résolution des chemins, primitives fichiers, boucle du Shell et parsing des commandes.
- Travail commun – Noyau et commandes :
structures de requêtes, fork(), table de dispatch, pointeurs de fonctions, implémentation des commandes et gestion des erreurs.
- Travail commun – IPC, Écran et validation :
pipes Shell ↔ Noyau, FIFO Shell → Écran, protocole d'échange, programme d'affichage, intégration et tests du système complet.

Nous suivrons l'ordre de développement suivant : structures communes → formatage du disque → allocation blocs/inodes → répertoires et resolve_path() → lecture/écriture → parser et structures de requêtes → table de dispatch → commandes simples → IPC Shell/Noyau → Écran → conteneurisation et persistance → commandes avancées → droits et finitions.